

The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style.

```
class TaskQueue:
    # A bounded first-in, first-out (FIFO) task
    queue.
    #
    # Tasks are appended to the end of the
    queue and processed in the
    # same order they were received. The queue
    enforces a maximum
    # capacity to prevent unbounded growth.
    #
    # max_size:
    #     Maximum number of tasks that may be
    stored at one time.
    #     Once this limit is reached,
    additional enqueue operations
    #     are rejected until space becomes
    available.
    def __init__(self, max_size=100):
        self.queue = []          # Ordered
        collection of pending tasks.
        self.max = max_size      # Maximum queue
        capacity.

    # Attempt to add a task to the back of the
    queue.
    #
    # The task is accepted only if the queue
    has not reached its
    # configured capacity.
    #
    # Returns:
    #     True - Task was successfully added.
    #     False - Queue is full and the task
    was rejected.
    #
```

```
# This method does not block or
automatically evict existing tasks
# when the queue is full.
def enqueue(self, task):
    if len(self.queue) >= self.max:
        return False
    self.queue.append(task)
    return True

# Remove and return the oldest task in the
queue.
#
# Because the queue follows FIFO semantics,
the task returned is
# always the one that has been waiting the
longest.
#
# Returns:
#     The next queued task if one exists.
#     None if the queue is currently empty.
#
# Returning None allows callers to handle
an empty queue without
# needing exception-based control flow.
def dequeue(self):
    if not self.queue:
        return None
    return self.queue.pop(0)

# Determine whether the queue currently
contains any tasks.
#
# Returns:
#     True if no tasks are waiting.
#     False if at least one task is
present.
#
# Useful for consumers that need to check
queue state before
# attempting to process work.
def is_empty(self):
    return len(self.queue) == 0
```

```
# Return the current number of queued
tasks.
#
# Returns:
#     Integer count of tasks currently
stored in the queue.
#
# This reflects the amount of pending work
waiting to be processed.
def size(self):
    return len(self.queue)

# Return the next task scheduled for
processing without removing it.
#
# Unlike dequeue(), this method leaves the
queue unchanged and
# provides a read-only view of the next
item.
#
# Returns:
#     The next task in FIFO order.
#     None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]
    return None

class RateLimiter:
    # Sliding-window rate limiter.
    #
    # The limiter tracks timestamps of recently
accepted requests and
    # determines whether a new request can be
accommodated within a
    # fixed time window.
    #
    # limit:
    #     Maximum number of requests permitted
during the window.
    #
    # window:
```

```
#      Duration of the evaluation window in
seconds.
#
# Example:
#      limit=100, window=60
#      Allows up to 100 requests during any
60-second period.
def __init__(self, limit, window):
    self.limit = limit
    self.window = window
    self.calls = []          # Timestamps of
accepted requests.

    # Evaluate whether a request occurring at
the supplied timestamp
    # should be allowed.
    #
    # Before checking capacity, timestamps that
fall outside the active
    # sliding window are discarded. The
remaining timestamps represent
    # requests that still count against the
current rate limit.
    #
    # If the number of retained requests is
below the configured limit,
    # the request is accepted, its timestamp is
recorded, and True is
    # returned.
    #
    # Otherwise the request is rejected and
False is returned.
    #
    # timestamp:
    #      Request time expressed in the same
units used to define the
    #      window (typically seconds since an
epoch).
    #
    # Returns:
    #      True if the request is allowed.
    #      False if accepting the request would
exceed the limit.
```

```
def allow(self, timestamp):
    # Remove requests that are no longer
    within the active
    # sliding window and therefore should
    not count toward
    # the current limit calculation.
    self.calls = [
        t for t in self.calls
        if timestamp - t < self.window
    ]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False


def retry(func, attempts=3, exceptions=
(Exception,)):
    # Execute a callable and automatically
    retry when specified
    # exceptions occur.
    #
    # This utility is useful for operations
    that may fail
    # transiently, such as network requests,
    external service calls,
    # or temporary resource contention.
    #
    # func:
    #     Callable to execute.
    #
    # attempts:
    #     Total number of execution attempts,
    including the initial
    #     invocation. A value of 3 means the
    function may be executed
    #     up to three times before failure is
    reported.
    #
    # exceptions:
    #     Tuple of exception types that should
```

```
trigger a retry.
#     Exceptions not matching these types
are propagated
#     immediately.
#
# Returns:
#     Whatever value is returned by func().
#
# Raises:
#     The final exception encountered if
all retry attempts fail.
#
# Note:
#     No delay or backoff strategy is
applied between retries.
    for i in range(attempts):
        try:
            return func()
        except exceptions:
            if i == attempts - 1:
                raise

# Defensive fallback.
#
# Under normal execution this line is
unreachable because the
# function either returns a successful
result or re-raises the
# final exception. It remains as a
safeguard against future
# modifications to the retry logic.
return None
```

The existing comments are already thorough, but they're somewhat repetitive and focus on describing *what* each line does. I would typically make them more API-oriented: explain intent, behavior, edge cases, performance characteristics where relevant, and return semantics without over-commenting obvious implementation details.

Here's how I would rewrite the documentation:

</> Python



▶ Run

```

class TaskQueue:
    """
    Fixed-capacity FIFO (first-in, first-out) queue for managing pending work.

    Tasks are appended to the tail of the queue and processed in the same
    order they were received. Once the configured capacity is reached,
    additional tasks are rejected rather than causing the queue to grow
    without bound.

    Args:
        max_size (int, optional):
            Maximum number of tasks that may be stored simultaneously.
            Defaults to 100.

    Notes:
        - enqueue() is non-blocking and does not evict existing tasks.
        - dequeue() returns None when the queue is empty.
        - This implementation uses a list internally; removing items from
          the front of the queue has O(n) time complexity.
    """

    def __init__(self, max_size=100):
        self.queue = []      # Pending tasks in FIFO order.
        self.max = max_size  # Queue capacity limit.

    def enqueue(self, task):
        """
        Add a task to the end of the queue.

        The task is accepted only if capacity remains available.

        Args:
            task:
                Object representing a unit of work.

        Returns:
            bool:
                True if the task was added successfully.
                False if the queue is already at capacity.
        """
        if len(self.queue) >= self.max:
            return False

        self.queue.append(task)
        return True

    def dequeue(self):
        """
        Remove and return the next task to be processed.

        Because the queue follows FIFO semantics, the returned task is
        always the one that has been waiting the longest.

```

```
    Returns:
        The next queued task, or None if the queue is empty.
    """
    if not self.queue:
        return None

    return self.queue.pop(0)

def is_empty(self):
    """
    Determine whether the queue currently contains any tasks.

    Returns:
        bool:
            True if the queue is empty.
            False otherwise.
    """
    return len(self.queue) == 0

def size(self):
    """
    Return the current number of queued tasks.

    Returns:
        int: Number of tasks waiting to be processed.
    """
    return len(self.queue)

def peek(self):
    """
    Return the next task without removing it from the queue.

    This method provides a read-only view of the task that would be
    returned by dequeue().

    Returns:
        The next queued task, or None if the queue is empty.
    """
    if self.queue:
        return self.queue[0]

    return None


class RateLimiter:
    """
    Sliding-window rate limiter.

    The limiter records timestamps for accepted requests and evaluates
    each new request against a rolling time window. A request is allowed
    only if accepting it would keep the total number of requests within
```

the configured limit.

Args:

limit (int):
Maximum number of requests allowed during the window.

window (float):
Window duration in seconds.

Example:

```
RateLimiter(limit=100, window=60)
```

Allows up to 100 requests during any rolling 60-second interval.

"""

```
def __init__(self, limit, window):  
    self.limit = limit  
    self.window = window  
    self.calls = []          # Timestamps of accepted requests.
```

```
def allow(self, timestamp):
```

"""

Determine whether a request should be accepted.

Before evaluating the limit, timestamps that fall outside the current sliding window are discarded. Only requests that remain within the active window contribute to the limit calculation.

If capacity is available, the supplied timestamp is recorded and the request is accepted.

Args:

timestamp:
Time of the request expressed in the same units used by the configured window (typically seconds).

Returns:

bool:
True if the request is allowed.
False if the rate limit has been reached.

"""

Retain only requests that still fall within the active window.

```
self.calls = [  
    t for t in self.calls  
    if timestamp - t < self.window  
]
```

```
if len(self.calls) < self.limit:  
    self.calls.append(timestamp)  
    return True
```

```
return False
```

```
def retry(func, attempts=3, exceptions=(Exception,)):
    """
    Execute a callable with automatic retry behavior.

    The callable is invoked repeatedly until it succeeds or the maximum
    number of attempts is exhausted. Only exceptions matching the
    supplied exception types trigger a retry; all other exceptions are
    propagated immediately.

    This pattern is commonly used for transient failures such as network
    interruptions, temporary service outages, or resource contention.

    Args:
        func (callable):
            Function to execute.

        attempts (int, optional):
            Maximum number of execution attempts, including the initial
            call. Defaults to 3.

        exceptions (tuple[type[Exception], ...], optional):
            Exception types that should trigger a retry. Defaults to
            (Exception,).

    Returns:
        Any:
            The value returned by func().

    Raises:
        Exception:
            Re-raises the final matching exception if all attempts fail.

    Notes:
        - No delay, jitter, or exponential backoff is applied.
        - The callable should be safe to execute multiple times.
        - Side effects may occur on every retry attempt.
    """
    for i in range(attempts):
        try:
            return func()

        except exceptions:
            if i == attempts - 1:
                raise

    # Defensive fallback. Under the current control flow this statement
    # is unreachable because execution either returns successfully or
```

```
# re-raises the final exception.  
return None
```

The main differences are:

1. **Docstring-style structure** (Args , Returns , Raises , Notes) rather than long prose blocks.
2. **Behavioral information** (FIFO guarantees, retry side effects, sliding-window semantics) is emphasized over line-by-line implementation details.
3. **Complexity and caveats** are documented where they're likely to matter (e.g., `list.pop(0)` being $O(n)$, retry idempotency concerns).
4. Comments are reduced to places where they explain intent rather than restating code.

